

Actividad de Aprendizaje 9 Tarea 5 Trabajo en equipo

Centro Universitario de Ciencias Exactas e Ingenierías

Departamento de Ciencias Computacionales

Sistemas Operativos - Sección: D09

Violeta del Rocío Becerra Velázquez

Valeria Catarina Gallegos Goncalves – 224786838

Arturo Garza Sánchez Villela – 225350731

Valeria Velázquez Velázquez - 224785718

Ingeniería en Computación

11/10/2026

Índice

Índice 1
Investigación 2
Preguntas 6
1. ¿En qué consiste el problema de la concurrencia? 6
2. ¿Cuáles son los procesos concurrentes cooperantes? 6
3. ¿En qué consiste la Exclusión mutua? 6
4. Defina Interbloqueo 6
5. Defina Inanición. 6
6. Defina Excesiva Cortesía. 6
7. ¿Qué son los Hilos? 6
8. ¿Qué son los Semáforos? 7
9. ¿Qué es lo que mejora el tener más de un núcleo? 7
Conclusión 7
Bibliografía 8

Investigación

Algoritmos de Dekker y Peterson: exclusión mutua entre dos procesos

Introducción

Cuando dos procesos comparten un recurso, hace falta algo que evite que lo usen al mismo tiempo. A esa zona de código se le llama sección crítica, y el problema de protegerla se conoce como el problema de la sección crítica. Si no se resuelve, pueden aparecer condiciones de carrera: un proceso trabaja con información desactualizada porque el otro modificó los datos compartidos justo mientras los leía.

Este trabajo presenta dos soluciones clásicas para dos procesos, el algoritmo de Dekker y el de Peterson. Se revisa su origen, su funcionamiento y sus diferencias, y se mencionan algunas alternativas.

Propiedades que debe cumplir una solución

Ambos algoritmos buscan cumplir tres propiedades:

- Exclusión mutua: si un proceso está en la sección crítica, ningún otro puede estar ahí al mismo tiempo.
- Progreso: si uno o más procesos quieren entrar, alguno debe poder hacerlo; no se bloquea a todos indefinidamente.
- Espera acotada: ningún proceso que esté esperando puede ser ignorado para siempre; tarde o temprano debe entrar.

Contexto histórico

El problema de coordinar el acceso a algo compartido es anterior a las computadoras. A finales del siglo XIX y principios del XX, el ferrocarril necesitaba evitar choques en vías compartidas, y lo resolvió con semáforos. Después, la telegrafía enfrentó un reto parecido al manejar varias transmisiones en canales limitados, y lo resolvió con multiplexores.

Con los sistemas multiprogramados de los años 60 y 70, el problema pasó a ser cómo administrar recursos computacionales compartidos. E. W. Dijkstra dedicó tres años a estudiarlo y más adelante introdujo los semáforos. El matemático T. J. Dekker formuló la primera solución exitosa. Aunque era correcta, resultaba muy compleja y difícil de entender. En 1981, Gary L. Peterson publicó el artículo “Myths About the Mutual Exclusion Problem”, donde presentó una solución más sencilla y a la vez formal. Su gran aportación fue demostrar que la exclusión mutua entre dos procesos se puede lograr con solo dos variables compartidas.

Algoritmo de Dekker

Definición: Es un algoritmo de programación concurrente que resuelve el problema de la sección crítica entre dos procesos. Es histórico por ser la primera solución correcta, y usa únicamente memoria compartida, con banderas booleanas y una variable de turno.

Condiciones de funcionamiento. Se asumen cuatro reglas:

- No hay jerarquías: ningún proceso tiene prioridad sobre el otro.
- El hardware es neutro: la velocidad o capacidad del equipo no importa.
- Hay tolerancia parcial a fallos: si un proceso "muere" fuera de la región crítica, el otro debe poder seguir.
- No se aceptan estancamientos: el bloqueo mutuo no es una solución válida.

Evolución en cinco versiones. El algoritmo se construyó por prueba y error:

- Versión 1 (alternancia estricta): los procesos se turnan obligatoriamente. El problema es que los acopla fuertemente, y un proceso lento retrasa a uno rápido.
- Versión 2 (interbloqueo): cambia los turnos por banderas. Si ambos procesos levantan su bandera a la vez, caen en deadlock y ninguno entra.
- Versión 3 (colisión e inactividad activa): intenta arreglar el interbloqueo bajando y subiendo banderas. No garantiza la exclusión mutua de forma eficiente o genera livelock, un ciclo infinito en el que ambos se ceden el paso.
- Versión 4 (postergación indefinida): añade retrasos. Evita el interbloqueo fijo, pero un proceso puede esperar un evento que tal vez nunca ocurra, lo que rompe la espera acotada.
- Versión 5 (solución definitiva): es el algoritmo de Dekker final. Combina la variable de turno de la versión 1 con las banderas de intención de las versiones siguientes.

Funcionamiento de la versión final. Si ambos procesos quieren entrar y las dos banderas están arriba, se revisa la variable de turno para desempatar. El proceso que no tiene el turno baja su bandera y espera pasivamente, lo que deja pasar al "ganador" sin conflictos. Al terminar, el ganador cede el turno.

Algoritmo de Peterson

Definición

Se trata de un algoritmo de coordinación entre procesos para garantizar la exclusión mutua entre ellos, es decir, que no puedan acceder a una sección crítica (recursos compartidos) al mismo tiempo.

Es de suma importancia pues la exclusión mutua entre dos procesos evita que existan condiciones de carrera, es decir, que uno de los procesos utilice información desactualizada porque en el momento en el que accedió a los datos compartidos, el otro proceso también lo hizo y los modificó, dejando con información errónea al otro proceso.

El algoritmo se basa en tres principios:

- Exclusión mutua: si un proceso se encuentra en una sección crítica, ningún otro proceso puede estarlo de manera simultánea.
- Progreso: si uno o más procesos desean entrar en una sección crítica, alguno de ellos debe de poder entrar, no se debe de bloquear a ninguno de los procesos de manera indefinida.
- Espera limitada: un proceso en espera de entrar a una sección crítica no debe de ignorarse para siempre, debe de llegar un momento en el que pueda entrar a la sección.

Para lograr este objetivo, Peterson propuso un algoritmo que, con el uso de sólo dos variables compartidas, puede decidir qué proceso entrará a la sección en cada momento.

El algoritmo de Peterson se basa en la cortesía, esto quiere decir que un proceso siempre prioriza que el otro entre primero antes que él.

Contexto histórico

Desde las décadas de los 60s y 70s, con del inicio de los sistemas multiprogramados, la importancia de coordinar procesos con recursos compartidos se volvió más y más esencial. Para ello, se estudiaron durante varios años diferentes tipos de soluciones para la exclusión mutua, algunos de los progresos más importante fueron hechos por Edsger Dijkstra que estudió diferentes soluciones y después introdujo los semáforos, posteriormente Teodorus Dekker introdujo el primer algoritmo para solucionar la exclusión mutua, sin embargo, era muy complejo y difícil de entender.

Siguiendo el trabajo de estos dos personajes, Gary L. Peterson en 1981 publicó un artículo titulado "Myths About the Mutual Exclusion Problem" en el cual explicaba su propio algoritmo de exclusión mutua, el cual resultó ser una respuesta mucho más sencilla y a la vez formal que la versión de Dekker. La gran aportación de Peterson fue demostrar que la exclusión mutua se podía lograr con apenas dos variables compartidas entre un par de procesos.

Funcionamiento

Se tienen dos variables compartidas entre los dos procesos a coordinar:

- Flag: el cual es un arreglo de booleanos que avisa si cualquiera de los algoritmos quiere entrar a la sección. Para decir que el proceso 0 quiere entrar a la sección se pone flag[0] en true, por otro lado si el proceso 1 quiere entrar a la sección, se pone flag[1] en true.
- Turn: se encarga de desempatar y decidir cuál de los dos procesos será el que entre primero a la sección crítica. Es un valor entero que tiene 0 si se le dará prioridad al proceso 0 y tiene 1 si se le dará prioridad al proceso 1.

La base del funcionamiento del algoritmo se base en que, cada vez que un algoritmo desee entrar a la sección crítica, debe de activar su flag y ceder el turno al otro proceso, por lo tanto, si el proceso 0 quiere entrar, el procedimiento que se seguirá será:

- Poner flag[0] en true
- Poner turn en 1

- Por otro lado, si el proceso 1 es el que desea entrar, se seguirán los siguientes pasos:
- Poner flag[1] en true
- Poner turn en 0

Una vez el proceso haya cedido su turno, espera de manera indefinida a que alguna de las siguientes cosas NO SE CUMPLA:

- Que el flag del otro proceso se encuentre encendido
- Que el turno diga que le toca al otro proceso

Mientras tanto, el proceso que haya sido elegido para entrar primero lo hará de inmediato y al salir, pondrá su propia bandera en false para que el siguiente proceso pueda entrar de inmediato.

Aplicando estos criterios, se puede generar la siguiente tabla para saber el comportamiento de un proceso para cada estado posible:

El otro quiere entrar	¿Es su turno?	¿Espero?
No	No	No
No	Si	No
Si	No	No
Si	Si	Si

Preguntas

1. ¿En qué consiste el problema de la concurrencia?

Cuando varios procesos corren al mismo tiempo y comparten recursos del sistema. Como no se puede saber con certeza en qué orden van a acceder a esos recursos, se pueden generar conflictos e inconsistencias, como condiciones de carrera, interbloqueos e inanición. Todo esto afecta la integridad de los datos y el buen funcionamiento del sistema.

2. ¿Cuáles son los procesos concurrentes cooperantes?

Son los procesos que trabajan juntos a propósito para lograr una meta en común. Para ello comparten datos, memoria u otros recursos, así que necesitan mecanismos de comunicación y sincronización que coordinen su ejecución y mantengan los resultados correctos y consistentes.

3. ¿En qué consiste la Exclusión mutua?

Es la regla que asegura que mientras un proceso esté dentro de su sección crítica, ningún otro puede entrar a esa misma sección. De esta forma, los recursos compartidos los usa un solo proceso a la vez y los datos no se corrompen.

4. Defina Interbloqueo

También llamado deadlock, ocurre cuando dos o más procesos se quedan bloqueados para siempre. Cada uno tiene un recurso que otro necesita y al mismo tiempo espera que los demás liberen los suyos. Se forma así una dependencia circular de la que ningún proceso puede salir sin ayuda externa.

5. Defina Inanición.

También llamada starvation, es cuando un proceso espera indefinidamente un recurso porque otros procesos, normalmente con mayor prioridad o trabajos más cortos, siempre pasan antes que él. El sistema sigue funcionando con normalidad, pero el proceso afectado nunca logra avanzar.

6. Defina Excesiva Cortesía.

Es un problema de sincronización en el que dos o más procesos se van cediendo el acceso a un recurso una y otra vez. Cada uno es tan “amable” con los demás que ninguno llega a usarlo. El resultado es una espera interminable parecida a la inanición, pero provocada por la propia lógica de cooperación.

7. ¿Qué son los Hilos?

Son la unidad de ejecución más pequeña dentro de un proceso. Comparten con los demás hilos del proceso el espacio de memoria y los recursos, pero cada uno tiene su propio contador de programa y su propia pila. Gracias a eso, un mismo proceso puede hacer varias tareas a la vez, lo que mejora el rendimiento y la capacidad de respuesta de la aplicación.

8. ¿Qué son los Semáforos?

Son variables enteras administradas por el sistema operativo que sirven para sincronizar procesos y controlar el acceso a recursos compartidos. Funcionan con dos operaciones atómicas: wait() resta uno al valor y bloquea al proceso si el resultado es negativo, signal() suma uno y si hay algún proceso esperando, lo despierta.

9. ¿Qué es lo que mejora el tener más de un núcleo?

Permite ejecutar varios procesos o hilos en paralelo de verdad, porque cada núcleo procesa instrucciones por su cuenta y al mismo tiempo. Esto se traduce en más rendimiento, menores tiempos de respuesta y mejor uso de los recursos, sobre todo en aplicaciones que dependen mucho de la concurrencia.

Conclusión

Como conclusión tenemos que el objetivo de manejar la concurrencia en los sistemas operativos es evitar que los procesos se estorben al compartir recursos. Problemas como las condiciones de carrera, el interbloqueo, la inanición o la excesiva cortesía demuestran que, si no se coordinan de forma adecuada, los datos se corrompen y el sistema se queda atascado.

Para resolver esto, soluciones históricas como los algoritmos de Dekker y Peterson demostraron que es posible garantizar la exclusión mutua, el progreso y la espera acotada utilizando únicamente memoria compartida. Mientras que Dekker logró una solución correcta después de varios intentos de prueba y error, Peterson simplificó el proceso utilizando la cortesía y tan solo dos variables (flag y turn) para desempatar y decidir quién entra a la sección crítica.

Hoy en día, estos mismos principios se aplican en herramientas más modernas como los semáforos y el uso de hilos. Al combinar estos mecanismos con procesadores de varios núcleos, el sistema logra ejecutar varias tareas al mismo tiempo de forma real, mejorando el rendimiento y la capacidad de respuesta sin perder la integridad de la información.

Bibliografía

1. GeeksforGeeks. (29 de Agosto de 2025). Dekker's algorithm in Process Synchronization. GeeksforGeeks. Recuperado el 30 de septiembre de 2026 de https://www.geeksforgeeks.org/dsa/dekkers-algorithm-in-process-synchronization/

2. Avril Chen. (2025). Dekker's algorithm. Avril Chen. Recuperado el 30 de septiembre de 2026 de https://avrilzchen.github.io/azchen/Dekker's-Algorithm

3. Mtro. José Luis Elvira. (2020, 14 junio). Sistemas Operativos, Problema de la concurrencia 6 Algoritmo de Dekker [Vídeo]. YouTube. Recuperado el 30 de septiembre de 2026 de https://www.youtube.com/watch?v=AxYF6H7VA2s

4. Mtro. José Luis Elvira. (2020b, junio 14). *Sistemas Operativos, Problema de la concurrencia 7 Algoritmo de Peterson* [Vídeo]. YouTube. Recuperado el 30 de septiembre de 2026 de https://www.youtube.com/watch?v=965y1o_5khY

5. Singh, V. (02 de abril *de 2025*). Algoritmo de Peterson en C. Medium. Recuperado el 30 de septiembre de 2026 de https://medium.com/@antiquark007/petersons-algorithm-in-c-57003757cada